

Validation of Medical Imaging AI

Day 2 - Model Performance Uncertainty Reporting

Worked solutions booklet

This handout contains solution code, computed results, and concise answers for Practicals 2.1-2.3. Tables for Practical 2.2 and 2.3 report bootstrap summaries only; redundant point-estimate columns are omitted.

Contents

Practical 2.1 - SD and SE of DSC

Practical 2.2 - Confidence intervals for individual model performance

Practical 2.3 - Confidence intervals for performance differences

Practical 2.1 - Standard Deviation and Standard Error of DSC

Two datasets of case-wise DSC values are used. Dataset A contains 25 cases, and Dataset B contains 1000 cases.

Solution code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

dataset_a = pd.read_csv("day2_practical_2_1_dataset_n25_left_skew.csv")
dataset_b = pd.read_csv("day2_practical_2_1_dataset_n1000_right_skew.csv")

def summarize_dsc(dataset, name):
    values = dataset["DSC"]
    n = len(values)
    sd = values.std(ddof=1)
    se = sd / np.sqrt(n)
    return {
        "Dataset": name,
        "n": n,
        "Mean DSC": values.mean(),
        "Median DSC": values.median(),
        "SD": sd,
        "SE": se,
    }

summary = pd.DataFrame([
    summarize_dsc(dataset_a, "Dataset A"),
    summarize_dsc(dataset_b, "Dataset B"),
])
summary
```

Results

Dataset	n	Mean DSC	Median DSC	SD	SE
Dataset A	25	0.800	0.819	0.075	0.015
Dataset B	1000	0.800	0.792	0.051	0.002

Answer

The two datasets have almost identical mean DSC values. The SD describes case-to-case variability, whereas the SE describes uncertainty in the estimated mean DSC. Dataset B has a much smaller SE because it contains many more test cases.

Practical 2.2 - Confidence Intervals for Individual Model Performance

Five algorithms are evaluated on four test sets. The proposed method is **alg_03**. For segmentation datasets, performance is mean DSC. For classification datasets, performance is Accuracy.

Solution code: performance tables

```
algorithms = ["alg_01", "alg_02", "alg_03", "alg_04", "alg_05"]
proposed_algorithm = "alg_03"

def segmentation_performance(dataset, algorithm):
    return dataset[algorithm].mean()

def classification_accuracy(dataset, algorithm):
    return (dataset[algorithm] == dataset["ground_truth"]).mean()

def format_mean_ci(mean_value, ci_lower, ci_upper):
    return f"{mean_value:.3f} ({ci_lower:.3f}, {ci_upper:.3f})"
```

Solution code: bootstrap confidence intervals

```
def bootstrap_mean_dsc(dataset, algorithm, n_bootstrap=1000, random_state=42):
    rng = np.random.default_rng(random_state)
    values = dataset[algorithm].values
    n = len(values)
    bootstrap_values = []

    for i in range(n_bootstrap):
        sample = rng.choice(values, size=n, replace=True)
        bootstrap_values.append(sample.mean())

    bootstrap_values = np.array(bootstrap_values)
    return (
        bootstrap_values.mean(),
        np.percentile(bootstrap_values, 2.5),
        np.percentile(bootstrap_values, 97.5),
    )

def bootstrap_accuracy(dataset, algorithm, n_bootstrap=1000, random_state=42):
    rng = np.random.default_rng(random_state)
    n = len(dataset)
    bootstrap_values = []

    for i in range(n_bootstrap):
        sample_indices = rng.choice(n, size=n, replace=True)
        sample = dataset.iloc[sample_indices]
        acc = (sample[algorithm] == sample["ground_truth"]).mean()
        bootstrap_values.append(acc)

    bootstrap_values = np.array(bootstrap_values)
    return (
        bootstrap_values.mean(),
        np.percentile(bootstrap_values, 2.5),
        np.percentile(bootstrap_values, 97.5),
    )
```

Practical 2.2 results

Final tables omit the redundant point-estimate column and report only the bootstrap mean with 95% CI.

Segmentation test set, n = 50

Algorithm	Mean DSC across bootstrap resamplings (95% CI)
alg_01	0.790 (0.777, 0.803)
alg_02	0.795 (0.782, 0.808)
alg_03 (proposed)	0.807 (0.793, 0.821)
alg_04	0.792 (0.779, 0.805)
alg_05	0.798 (0.785, 0.811)

Segmentation test set, n = 5000

Algorithm	Mean DSC across bootstrap resamplings (95% CI)
alg_01	0.798 (0.796, 0.800)
alg_02	0.801 (0.799, 0.803)
alg_03 (proposed)	0.822 (0.820, 0.824)
alg_04	0.800 (0.798, 0.802)
alg_05	0.803 (0.801, 0.805)

Binary classification test set, n = 20

Algorithm	Mean Accuracy across bootstrap resamplings (95% CI)
alg_01	0.800 (0.600, 0.950)
alg_02	0.796 (0.600, 0.950)
alg_03 (proposed)	0.850 (0.650, 1.000)
alg_04	0.753 (0.550, 0.950)
alg_05	0.796 (0.600, 0.950)

Multiclass classification test set, n = 1000

Algorithm	Mean Accuracy across bootstrap resamplings (95% CI)
alg_01	0.800 (0.776, 0.825)
alg_02	0.802 (0.776, 0.824)
alg_03 (proposed)	0.822 (0.798, 0.845)
alg_04	0.798 (0.773, 0.824)
alg_05	0.804 (0.781, 0.828)

Answer

The larger test sets have narrower confidence intervals. Small test sets produce wider intervals, so a marginally higher model performance estimate should be interpreted cautiously.

Practical 2.3 - Confidence Intervals for Performance Differences

The comparison of interest is the proposed method against each comparator. The bootstrap is therefore applied to the performance difference, not to each individual method separately.

Solution code: bootstrap differences

```
comparators = [alg for alg in algorithms if alg != proposed_algorithm]

def bootstrap_difference_mean_dsc(dataset, comparator, n_bootstrap=1000, random_state=42):
    rng = np.random.default_rng(random_state)
    n = len(dataset)
    bootstrap_differences = []

    for i in range(n_bootstrap):
        sample_indices = rng.choice(n, size=n, replace=True)
        sample = dataset.iloc[sample_indices]
        proposed_perf = sample[proposed_algorithm].mean()
        comparator_perf = sample[comparator].mean()
        bootstrap_differences.append(proposed_perf - comparator_perf)

    bootstrap_differences = np.array(bootstrap_differences)
    return (
        bootstrap_differences.mean(),
        np.percentile(bootstrap_differences, 2.5),
        np.percentile(bootstrap_differences, 97.5),
    )

def bootstrap_difference_accuracy(dataset, comparator, n_bootstrap=1000, random_state=42):
    rng = np.random.default_rng(random_state)
    n = len(dataset)
    bootstrap_differences = []

    for i in range(n_bootstrap):
        sample_indices = rng.choice(n, size=n, replace=True)
        sample = dataset.iloc[sample_indices]
        proposed_perf = (sample[proposed_algorithm] == sample["ground_truth"]).mean()
        comparator_perf = (sample[comparator] == sample["ground_truth"]).mean()
        bootstrap_differences.append(proposed_perf - comparator_perf)

    bootstrap_differences = np.array(bootstrap_differences)
    return (
        bootstrap_differences.mean(),
        np.percentile(bootstrap_differences, 2.5),
        np.percentile(bootstrap_differences, 97.5),
    )
```

Practical 2.3 results

Final tables report only the mean performance difference across bootstrap resamplings with 95% CI.

Segmentation test set, n = 50

Comparison	Mean difference (95% CI)
alg_03 - alg_01	0.017 (0.011, 0.023)
alg_03 - alg_02	0.012 (0.005, 0.019)
alg_03 - alg_04	0.015 (0.008, 0.022)
alg_03 - alg_05	0.009 (0.003, 0.016)

Segmentation test set, n = 5000

Comparison	Mean difference (95% CI)
alg_03 - alg_01	0.024 (0.023, 0.025)
alg_03 - alg_02	0.021 (0.020, 0.022)
alg_03 - alg_04	0.022 (0.021, 0.023)
alg_03 - alg_05	0.019 (0.018, 0.020)

Binary classification test set, n = 20

Comparison	Mean difference (95% CI)
alg_03 - alg_01	0.050 (-0.150, 0.250)
alg_03 - alg_02	0.054 (-0.200, 0.300)
alg_03 - alg_04	0.098 (-0.150, 0.300)
alg_03 - alg_05	0.054 (-0.200, 0.300)

Multiclass classification test set, n = 1000

Comparison	Mean difference (95% CI)
alg_03 - alg_01	0.022 (-0.011, 0.056)
alg_03 - alg_02	0.021 (-0.012, 0.055)
alg_03 - alg_04	0.024 (-0.012, 0.058)
alg_03 - alg_05	0.018 (-0.014, 0.053)

Answer

For model comparison, the uncertainty should be computed for the difference itself. A confidence interval for the difference that excludes zero provides evidence of a performance difference. Overlap between individual model CIs is not a reliable substitute for this analysis.

Summary

Concept	Interpretation
SD	Variability of performance between individual cases.
SE	Uncertainty in the estimated mean performance.
Performance CI	Uncertainty around one model performance estimate.
Difference CI	Uncertainty around the comparison between the proposed method and a comparator.
Main principle	When comparing algorithms, estimate uncertainty for the performance difference directly.

Main takeaway

A reported model performance value is incomplete without uncertainty. When the goal is to compare algorithms, the most relevant quantity is the performance difference between models and its confidence interval.